GUIA PARA EXPONER

Como despliegue CampusFlow

La ruta desde el navegador hasta la API, el VPS, Docker, Nginx, HTTPS y GitHub Actions.



Una explicacion clara, en primera persona y basada en la configuracion real del servidor.

CAMPUSFLOW · DESPLIEGUE VPS

Rama: refactor

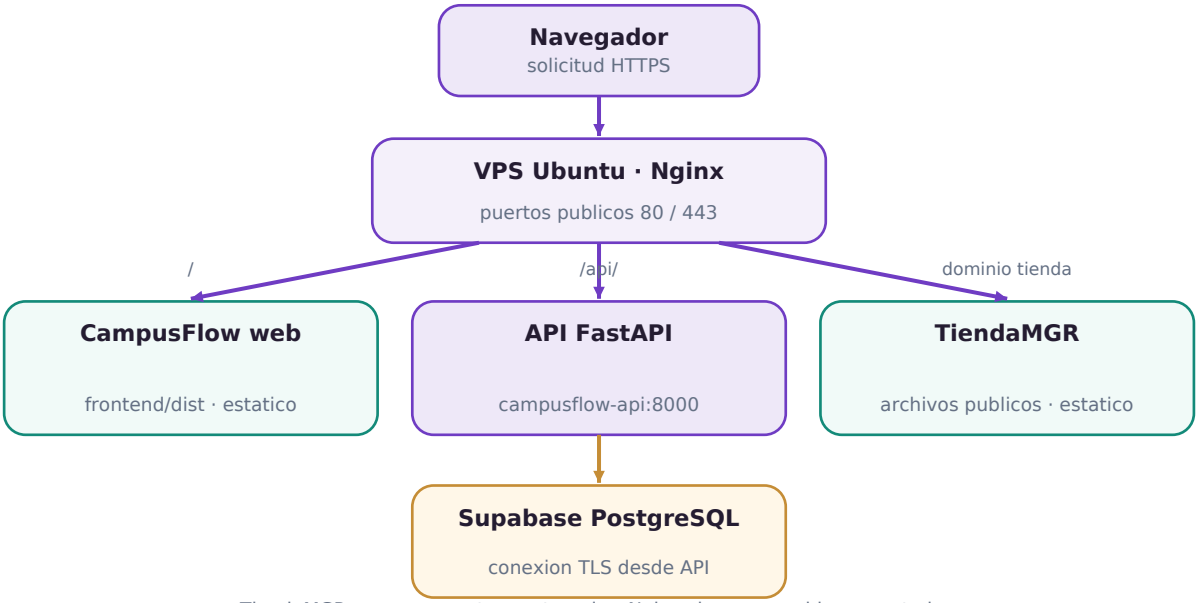
VPS Ubuntu en DigitalOcean

Verificacion practica: reinicio del servidor y comprobacion de los sitios

25 de octubre de 2026

Guia personal para explicar y repetir el despliegue

1. La arquitectura en una vista



TiendaMGR no corre en otro contenedor: Nginx sirve sus archivos montados.

Cuando alguien abre uno de mis sitios, el dominio de DuckDNS resuelve a la IP pública del VPS. El VPS recibe la conexión en los puertos 80 o 443. El único contenedor que publica esos puertos es Nginx.

Hay una precision que conviene decir correctamente: actualmente los contenedores activos son `campusflow-nginx` y `campusflow-api`. TiendaMGR no tiene un contenedor de aplicacion activo en esta configuracion; Nginx sirve sus archivos estaticos desde una carpeta montada. Antes habia un contenedor Caddy para esa tienda, pero ahora los dos dominios pasan por el mismo Nginx.

2. Que es Docker y que es Nginx

Docker me permite empaquetar una aplicacion con su entorno para ejecutarla de forma aislada. Una **imagen** es el paquete que construyo; un **contenedor** es una instancia de esa imagen que esta corriendo. Por ejemplo, la imagen `campusflow-api:refactor` inicia FastAPI con Uvicorn.

Docker Compose describe los servicios que deben existir, sus redes, volumenes, variables y reglas de reinicio. En este proyecto, `deploy/compose.vps.yaml` define la API y Nginx. Los dos se conectan a la red Docker externa `entrada_web`.

Nginx es el punto de entrada web. En este proyecto hace dos trabajos:

- **Servidor de archivos estaticos:** entrega el frontend compilado de CampusFlow y los archivos publicos de TiendaMGR.
- **Proxy inverso:** recibe una solicitud a `/api/` y la reenvia al servicio `campusflow-api` dentro de la red Docker.

El nombre `campusflow-api` funciona como nombre de red entre contenedores. Nginx no necesita saber una IP interna fija: Docker resuelve el nombre al contenedor correcto.

3. Como viaja una peticion

1. El navegador busca el dominio en DNS. DuckDNS apunta ambos nombres a la IP publica del VPS.
2. Si el navegador llega por HTTP al puerto 80, Nginx responde con una redireccion `301` hacia HTTPS. La excepcion es la ruta temporal que Certbot usa para validar el dominio.
3. El navegador abre HTTPS por el puerto 443. Nginx presenta el certificado correspondiente al dominio.
4. Nginx revisa el dominio y la ruta solicitada.
5. Si es una pagina de CampusFlow, entrega los archivos de `frontend/dist`. Si es una ruta `/api/`, reenvia la solicitud al contenedor FastAPI por la red privada de Docker.
6. FastAPI procesa la ruta y, cuando necesita datos, consulta PostgreSQL en Supabase. La respuesta regresa por el mismo camino hasta el navegador.

En el Compose de produccion, la API tiene `expose: 8000`, pero no publica ese puerto en Internet. Solo Nginx expone los puertos 80 y 443 del VPS. Asi no dejo la API accesible directamente desde fuera.

La ruta `/api/docs` es una forma sencilla de comprobar el proxy: Nginx recibe `/api/docs`, quita el prefijo `/api/` de acuerdo con su `proxy_pass` y consulta `/docs` en FastAPI.

4. Como funciona HTTPS y TLS

HTTPS es HTTP protegido mediante TLS. TLS cifra la comunicacion entre el navegador y el servidor y permite verificar que el certificado corresponde al dominio visitado. En mi configuracion, Nginx termina TLS: el navegador negocia una conexion segura con Nginx en el puerto 443.

Use Let's Encrypt para obtener un certificado que cubre los dos dominios. Certbot valida que controlo los dominios mediante el desafio HTTP de ACME, que Nginx deja pasar en `/.well-known/acme-challenge/`. Nginx carga el certificado publico `fullchain.pem` y la llave privada `privkey.pem` desde el volumen de certificados.

Las solicitudes normales por HTTP se redirigen a HTTPS. El trafico entre Nginx y FastAPI viaja por la red privada de Docker usando HTTP; el cifrado TLS termina en Nginx. La conexion de la API con PostgreSQL se configura aparte con TLS requerido.

La renovacion se automatiza con una tarea de cron que ejecuta `certbot renew` y luego recarga Nginx. Asi no tengo que renovar el certificado manualmente cada vez.

5. Como funciona mi CI/CD

El workflow esta en `.github/workflows/desplegar-refactor.yml`. Se activa al subir cambios a la rama `refactor` o al iniciarlo manualmente desde GitHub Actions. No despliega `main`.

Etapas del pipeline

1. **Obtener codigo:** GitHub Actions descarga el commit de `refactor`.
2. **Compilar frontend:** prepara Node 22, instala las dependencias con `npm ci` y compila con `npm run build`.
3. **Construir API:** crea la imagen Docker de FastAPI.
4. **Empaquetar:** guarda la imagen y los archivos compilados del frontend como archivos comprimidos.
5. **Conectarse con seguridad:** usa una llave SSH guardada como secreto y verifica la identidad del VPS con `known_hosts`. No guarda la llave privada en el repositorio.
6. **Transferir:** copia los paquetes al usuario dedicado `campusflow-deploy` en el VPS.
7. **Desplegar:** el servidor actualiza el codigo al commit exacto, guarda copias de la version anterior, carga la nueva imagen y reemplaza los archivos del frontend.
8. **Iniciar servicios:** Docker Compose recrea la API y Nginx con los archivos nuevos.
9. **Verificar:** el pipeline prueba Nginx y consulta la pagina principal y `/api/docs` por HTTPS.
10. **Revertir si falla:** si la verificacion no funciona, el workflow restaura la imagen, configuracion y frontend anteriores.

La diferencia entre CI y CD en este caso es que **CI** prepara y compila el cambio, y **CD** lo entrega al VPS y verifica que responda. El workflow actual compila, pero no tiene una etapa de pruebas automatizadas ni ejecuta migraciones de Alembic. Antes de desplegar cambios de base de datos, debo planear y ejecutar la migracion de forma segura.

6. Como vuelve a iniciar despues de reiniciar el VPS

Docker esta habilitado como servicio de Ubuntu, por lo que el sistema lo inicia durante el arranque. Los servicios `campusflow-api` y `campusflow-nginx` usan la politica `restart: unless-stopped`. Si el VPS se reinicia, Docker vuelve a iniciar esos contenedores automaticamente, salvo que yo los haya detenido a proposito.

Tambien significa que si el proceso principal de un contenedor termina, Docker intenta iniciarlo de nuevo. Esta politica ayuda con fallos del proceso, pero no reemplaza un chequeo de salud: el Compose actual no define un `healthcheck` para FastAPI.

Prueba que hice

Reinicie el VPS el 25 de octubre de 2026. No ejecute comandos para iniciar manualmente las aplicaciones despues del reinicio. Docker inicio los dos contenedores. Luego confirme que CampusFlow, TiendaMGR y la documentacion de la API respondian con HTTP `200`.

En la primera consulta inmediata, la API todavia estaba terminando de arrancar y Nginx respondio `502` en esa ruta. Al terminar el inicio de FastAPI, la misma comprobacion respondio `200`. Esto muestra que los contenedores se recuperan solos, y tambien que conviene agregar un `healthcheck` y una verificacion de disponibilidad durante el arranque.

7. Si vuelvo a desplegar otra aplicacion Docker

Puedo usar estos archivos como referencia, adaptandolos. No debo copiarlos sin cambiar nombres y rutas porque estan hechos para este VPS y este proyecto.

Archivo de referencia	Para que sirve	Que debo adaptar
`deploy/compose.vps.yaml`	Servicios, red, volumenes y politica de reinicio	Nombre de imagen, contenedor, puerto, rutas y variables
`deploy/nginx-api.conf.example`	Ejemplo de Nginx para un dominio de API	Dominio, certificado, destino y rutas del proxy
`deploy/nginx-campusflow.conf`	Configuracion real de los dos dominios actuales	No modificarla sin comprobar ambos sitios
`.github/workflows/desplegar-refactor.yml`	Ejemplo real del pipeline de compilacion y despliegue	Rama, pasos de build, usuario, rutas, pruebas y rollback

Lista antes de publicar

- Definir dominio, DNS, puertos y quien sera el unico proxy que ocupa 80 y 443.
- Confirmar que el servidor tiene RAM y disco suficientes, y revisar que otros servicios ya usan recursos.
- Crear Dockerfile reproducible y fijar versiones base. No poner claves dentro de la imagen.
- Decidir que datos necesitan persistir en volumenes y que datos viven en una base de datos externa.
- Separar variables publicas de secretos. Guardar llaves privadas y contrasenas en secretos de GitHub o en el `.env` protegido del VPS, nunca en Git.
- Crear y verificar una red Docker; publicar solo los puertos que realmente deban recibir trafico externo.
- Configurar TLS antes de enviar trafico real y verificar la renovacion del certificado.
- Agregar chequeos de salud, pruebas automatizadas, copias de seguridad y un plan de rollback.
- Si el cambio altera la base de datos, preparar la migracion y el respaldo antes del despliegue. El pipeline actual no aplica Alembic automaticamente.
- Hacer un despliegue de prueba y revisar logs, pagina, API, base de datos y archivos persistentes.

8. Antes de probar un pago de Paddle Sandbox

El despliegue y el pago son pasos distintos. Para una prueba de Sandbox tengo que confirmar que:

- El entorno sea `sandbox`, con una API key Sandbox y un token de cliente que empiece con `test_`.
- El precio exista y este activo en el mismo catalogo Sandbox.
- El enlace de pago predeterminado use mi dominio y el dominio este aprobado en Paddle.
- El destino del webhook use HTTPS y tenga configurados los eventos requeridos.
- `PADDLE_WEBHOOK_SECRET` sea el secreto de firma de esa notificacion, no una API key.
- El acceso se active solo despues del webhook valido; la pagina de retorno por si sola no confirma un pago.

Sandbox no cobra dinero real. Para una compra de prueba exitosa se puede usar la tarjeta oficial de Paddle `4242 4242 4242 4242`, cualquier nombre, vencimiento futuro y CVV `100`. Luego reviso en Paddle que la transaccion se completo, que llegaron los webhooks y que la institucion quedo activa. Para probar un rechazo se usa la tarjeta de prueba que Paddle documenta como declinada.

Una respuesta corta para exponer

Yo subi CampusFlow a un VPS con Docker. Nginx es la puerta de entrada: atiende los dominios, redirige HTTP a HTTPS y sirve el frontend. Cuando la ruta empieza con `/api`, Nginx la pasa por una red privada de Docker al contenedor de FastAPI. La tienda se entrega como archivos estaticos desde el mismo Nginx. Let's Encrypt proporciona el certificado TLS y Certbot lo renueva automaticamente. GitHub Actions compila el frontend, construye la imagen de la API, la envia al VPS por SSH, actualiza los servicios y comprueba que respondan. Docker esta habilitado al iniciar Ubuntu y los contenedores tienen `unless-stopped`; por eso, despues de reiniciar el servidor, volvieron a levantarse sin que yo iniciara nada a mano.

Preguntas que me pueden hacer

Que pasa si el usuario escribe HTTP? Nginx responde con una redireccion `301` al mismo dominio por HTTPS.

La API esta expuesta directamente a Internet? No. El Compose la expone solo dentro de la red Docker en el puerto 8000. El navegador llega a ella a traves de Nginx.

Que hace el certificado TLS? Cifra el tramo entre el navegador y Nginx y permite verificar que el sitio corresponde al dominio.

Que pasa si el despliegue nuevo falla? El workflow intenta restaurar la imagen y los archivos anteriores si fallan sus comprobaciones finales.

Se ejecutan pruebas y migraciones en cada push? No en el workflow actual. Compila, despliega y comprueba que la web y la API respondan. Las pruebas y migraciones automaticas son mejoras que debo agregar antes de depender de ellas.